МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 12

Тема: Поведенческие паттерны: Архитектурные паттерны

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы архитектурного паттерна проектирования MVC (Model-View-Controller) и реализовать прикладную задачу «Графический пароль» с использованием данного паттерна.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать один из архитектурных паттернов: MVC, MVP, MVVM или Clean Architecture.

В соответствии с заданием был выбран паттерн MVC (Model-View-Controller).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль. Архитектура приложения разделена на три компонента: Model (данные и логика), View (отображение), Controller (связующее звено).

1. Описание паттерна MVC

Паттерн MVC (Model-View-Controller) - это архитектурный паттерн проектирования, который разделяет приложение на три слабосвязанных компонента:

- Model - данные и бизнес-логика приложения;
- View - отображение данных пользователю;
- Controller - обработка пользовательского ввода и связывание Model с View.

Данный паттерн применяется в ситуациях, когда нужно отделить бизнес-логику от пользовательского интерфейса, чтобы обеспечить независимость компонентов, упростить тестирование и повторное использование.

В разработанном приложении паттерн MVC используется следующим образом:

- Model (PasswordModel) - хранит выбранную последовательность и хэш пароля, реализует операции создания, проверки и сброса пароля, работает с файлом graphical_password.json. Не зависит от Tkinter.

- View (PasswordView) - строит интерфейс и отображает данные. Отвечает только за визуализацию (сетка, кнопки, метки, диалоговые окна). Не содержит бизнес-логики.

- Controller (PasswordController) - принимает события от View, вызывает методы Model и обновляет View. Координирует работу двух других компонентов.

Поток данных:

1. Пользователь нажимает ячейку.

2. View вызывает controller.on_cell_clicked(number).

3. Controller вызывает model.select_cell(number).

4. Controller вызывает view.render(...), чтобы обновить отображение.

Благодаря такому разделению, изменение UI не затрагивает логику, а изменение логики не требует правок в UI.

2. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Основное окно содержит:

- сетку 3×3 из девяти скруглённых ячеек;

- текущую последовательность;

- основную кнопку («Войти» / «Сохранить пароль»);

- кнопку «Очистить»;

- переключатель режимов «Вход» / «Создать пароль»;

- кнопку «Сбросить сохранённый пароль»;

- статусную строку.

К паролю предъявляются требования:

- длина от 4 до 9 ячеек;

- отсутствие повторов;

- ячейки в диапазоне 1–9.

3. Диаграмма классов

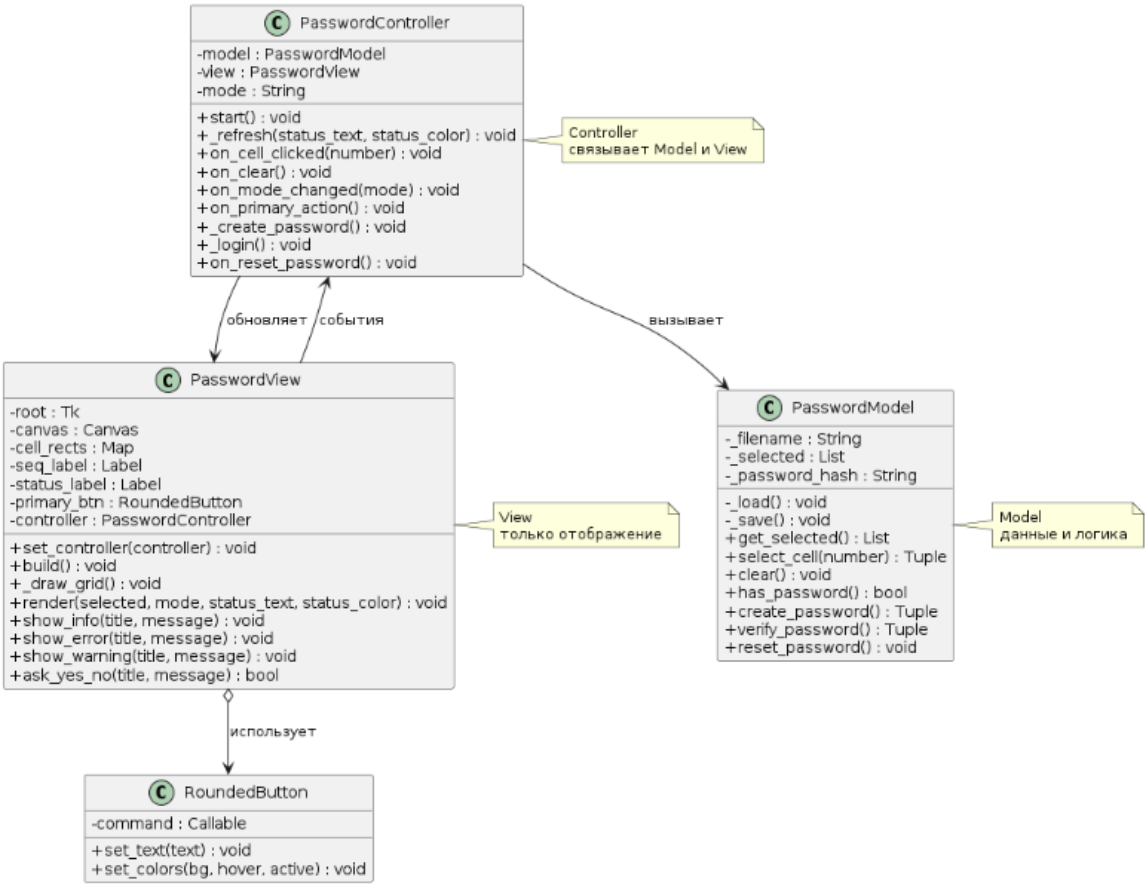


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит три класса архитектуры MVC и вспомогательные элементы.

4. Реализация паттерна MVC

Model - PasswordModel

```
class PasswordModel:
    def __init__(self):
        self._filename = "graphical_password.json"
        self._selected = []
        self._password_hash = None
        self._load()

    def select_cell(self, number):
        if number in self._selected:
            return False, "Эта ячейка уже выбрана."
```

```
        if len(self._selected) >= 9:
            return False, "Максимальная длина - 9."
        self._selected.append(number)
        return True, ""

    def create_password(self):
        ok, msg = validate_sequence(self._selected)
        if not ok:
            return False, msg
        self._password_hash = hash_sequence(self._selected)
        self._save()
        return True, "Графический пароль успешно создан."

    def verify_password(self):
        if not self.has_password():
            return False, "Пароль не создан. Сначала создайте пароль."
        ok, msg = validate_sequence(self._selected)
        if not ok:
            return False, msg
        if hash_sequence(self._selected) == self._password_hash:
            return True, "Пароль верный. Доступ разрешён."
        return False, "Пароль неверный. Доступ запрещён."
```

Model не импортирует tkinter и не знает о существовании UI.

View - PasswordView

python

```
class PasswordView:
    def set_controller(self, controller):
        self.controller = controller
```

```
    def build(self):
        ...
        self.primary_btn = RoundedButton(
            self.root, text="Войти",
            command=lambda: self.controller.on_primary_action(),
            ...
        )

    def render(self, selected, mode, status_text="", status_color="#ffffff"):
        for i, rect_id in self.cell_rects.items():
            color = "#6a6a6a" if i in selected else "#4a4a4a"
            self.canvas.itemconfig(rect_id, fill=color)
        ...
        self.status_label.config(text=status_text, fg=status_color)
```

View только отображает данные, полученные от Controller, и передаёт события обратно в Controller.

Controller - PasswordController

```
class PasswordController:
    def __init__(self, model, view):
        self.model = model
        self.view = view
        self.mode = "login"
        self.view.set_controller(self)

    def on_cell_clicked(self, number):
        ok, msg = self.model.select_cell(number)
        if ok:
            self._refresh()
        else:
            self._refresh(msg, "#ffaa00")
```

```
    def on_primary_action(self):
        if self.mode == "login":
            self._login()
        else:
            self._create_password()

    def _refresh(self, status_text="", status_color="#ffffff"):
        self.view.render(
            self.model.get_selected(),
            self.mode,
            status_text,
            status_color
        )
```

Controller - единственное место, где Model и View «встречаются».

Точка входа

```
if __name__ == "__main__":
    root = tk.Tk()
    model = PasswordModel()
    view = PasswordView(root)
    controller = PasswordController(model, view)
    controller.start()
    root.mainloop()
```

5. Результаты работы программы

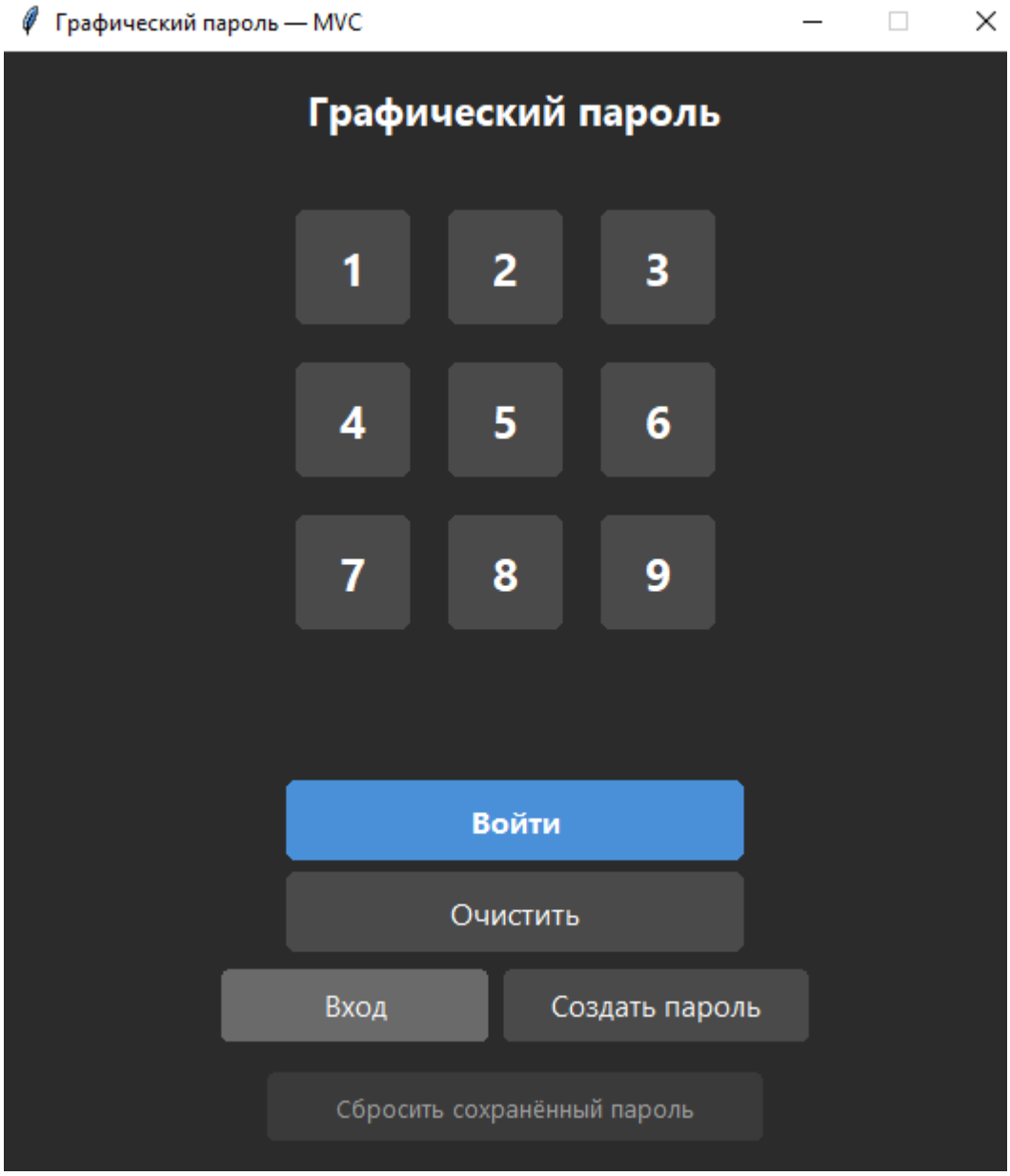


Рисунок 2 - Главное окно приложения

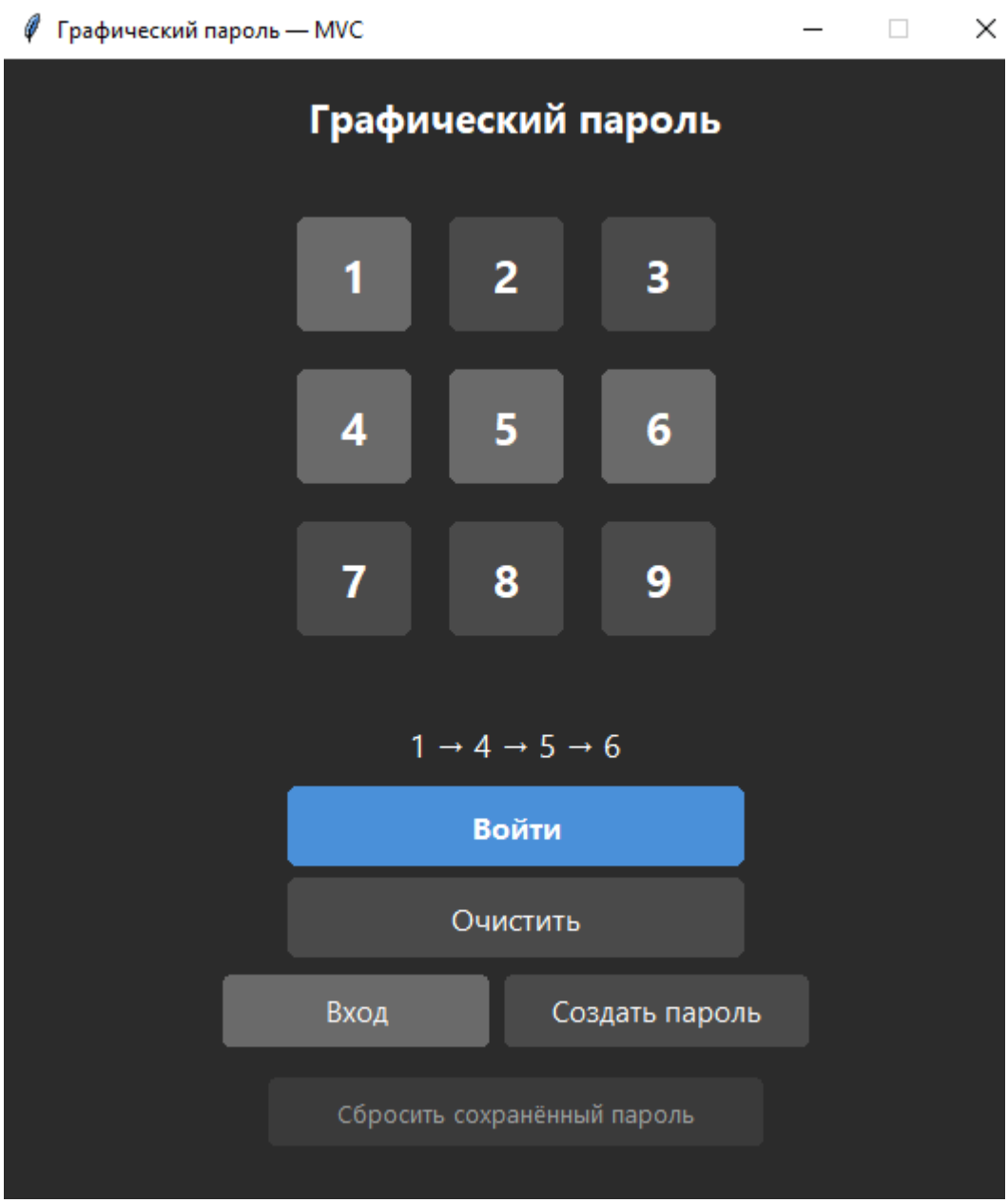


Рисунок 3 - Формирование последовательности

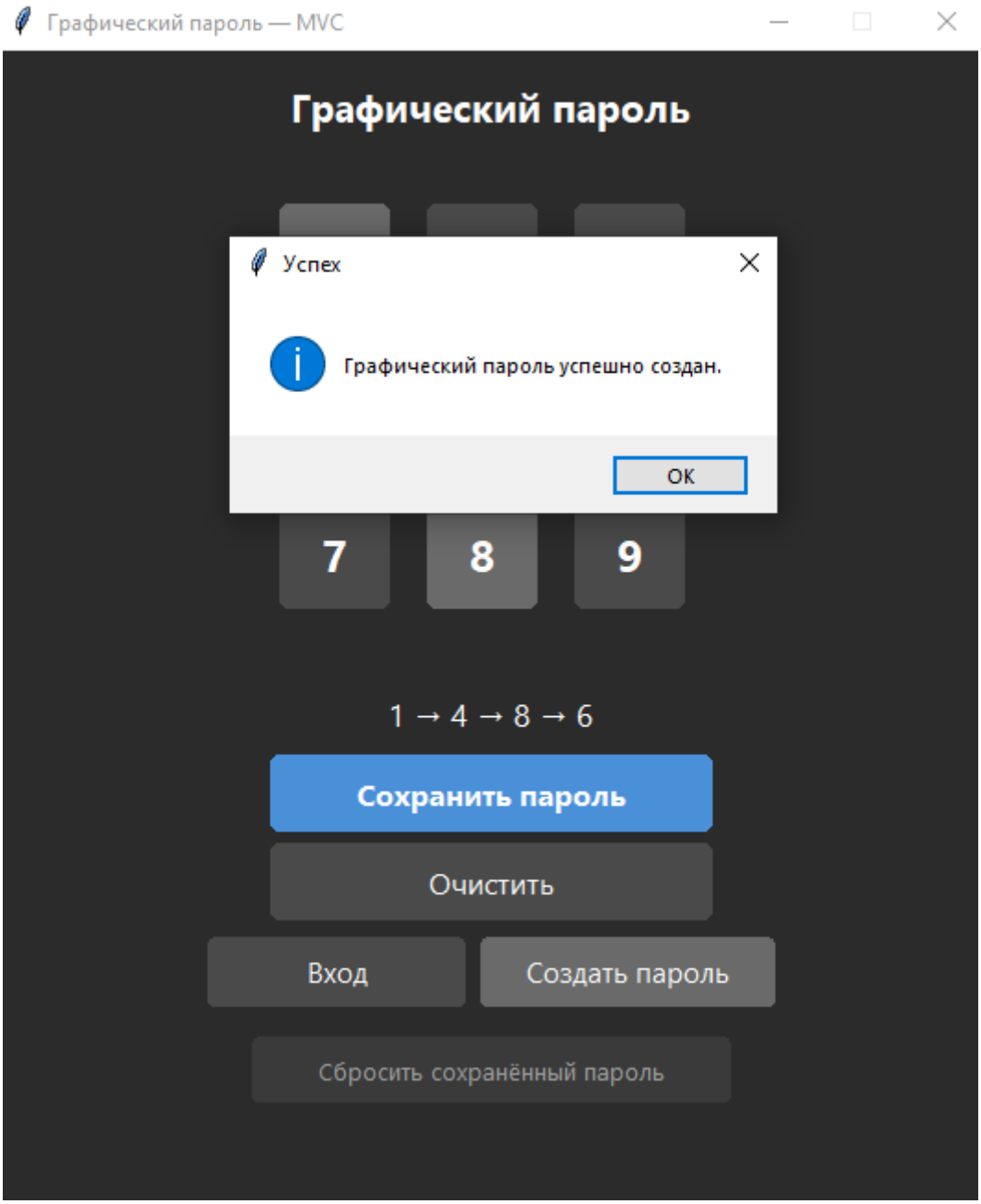


Рисунок 4 - Создание пароля

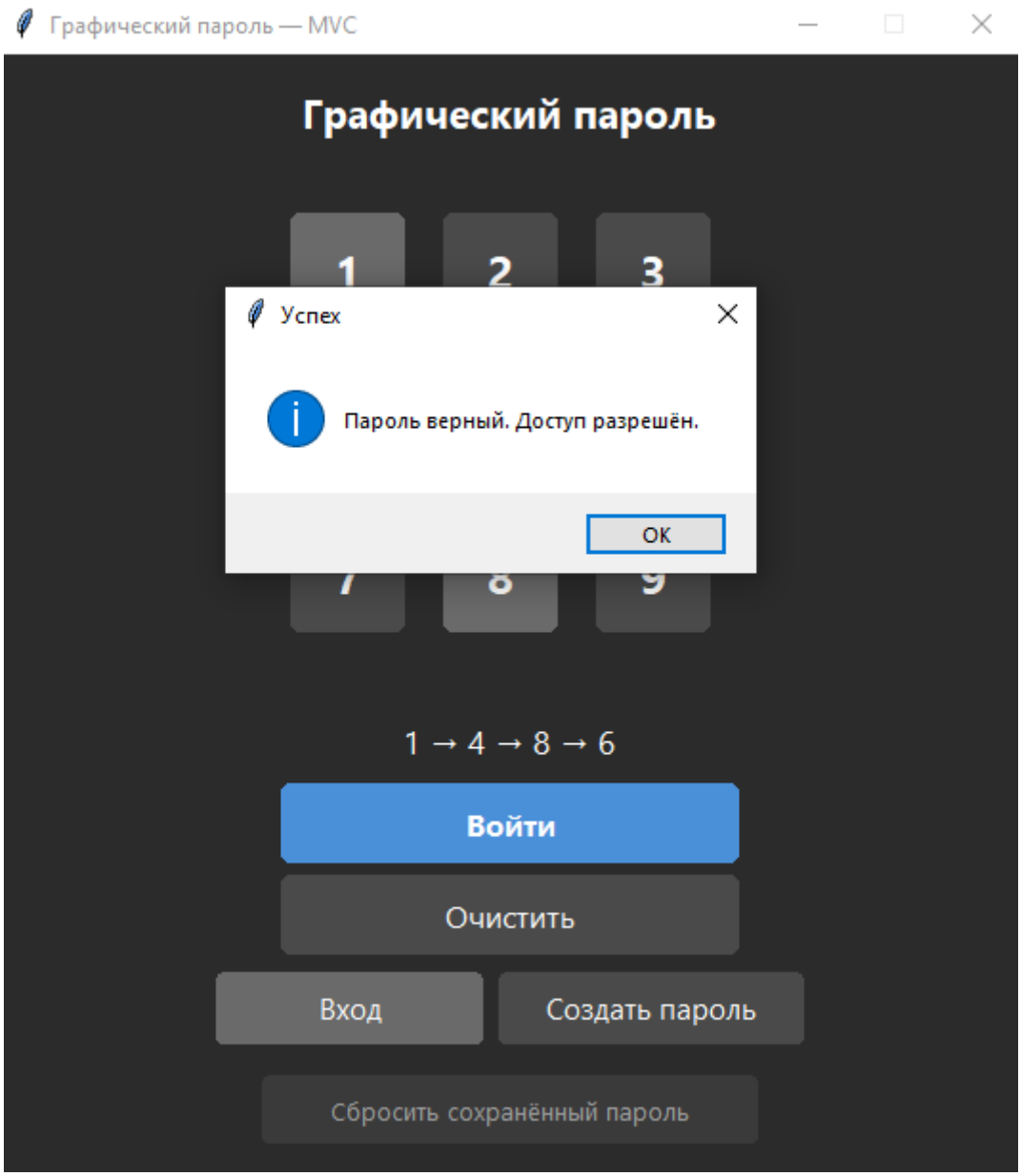


Рисунок 5 - Проверка правильного пароля

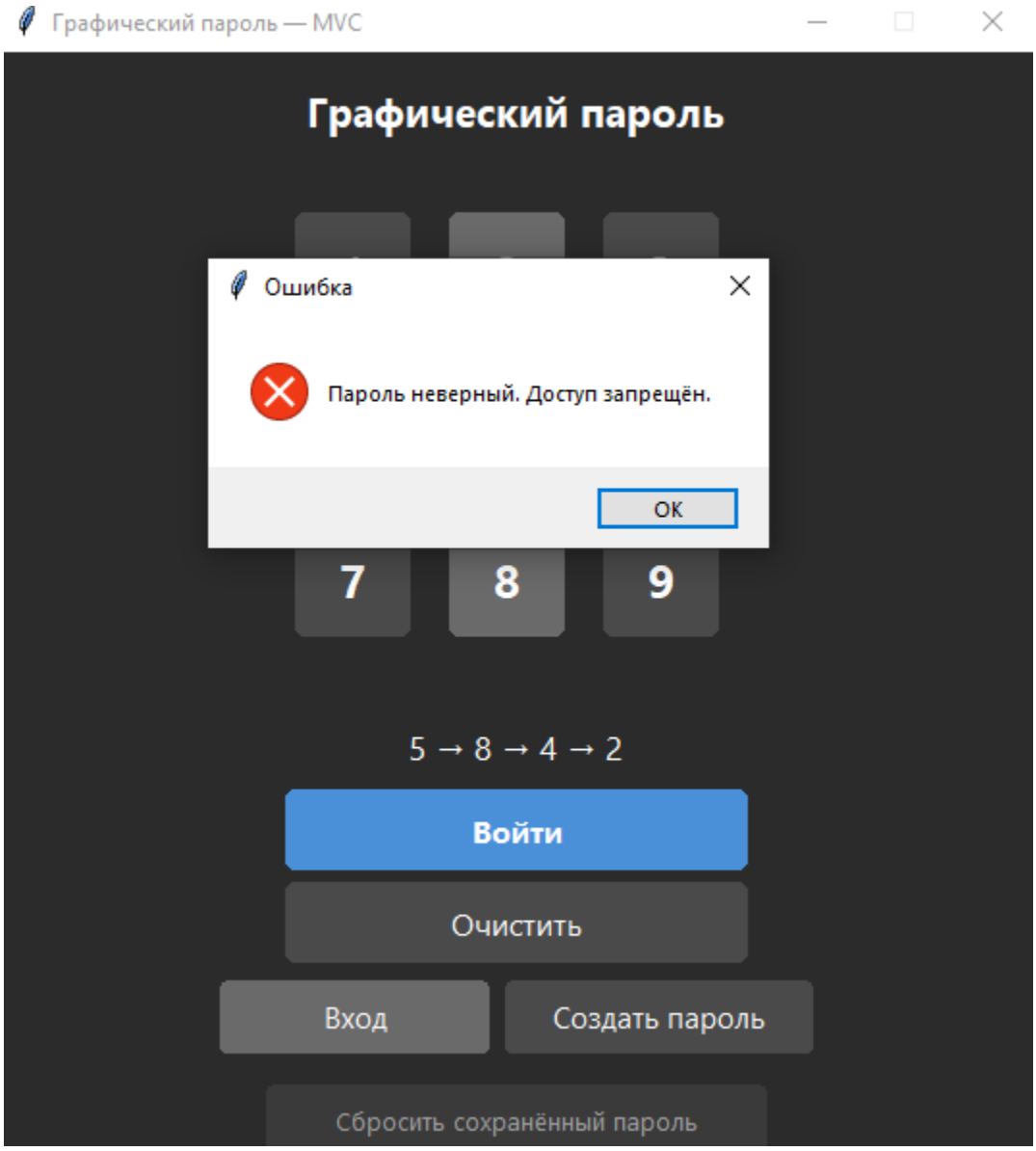


Рисунок 6 - Проверка неправильного пароля

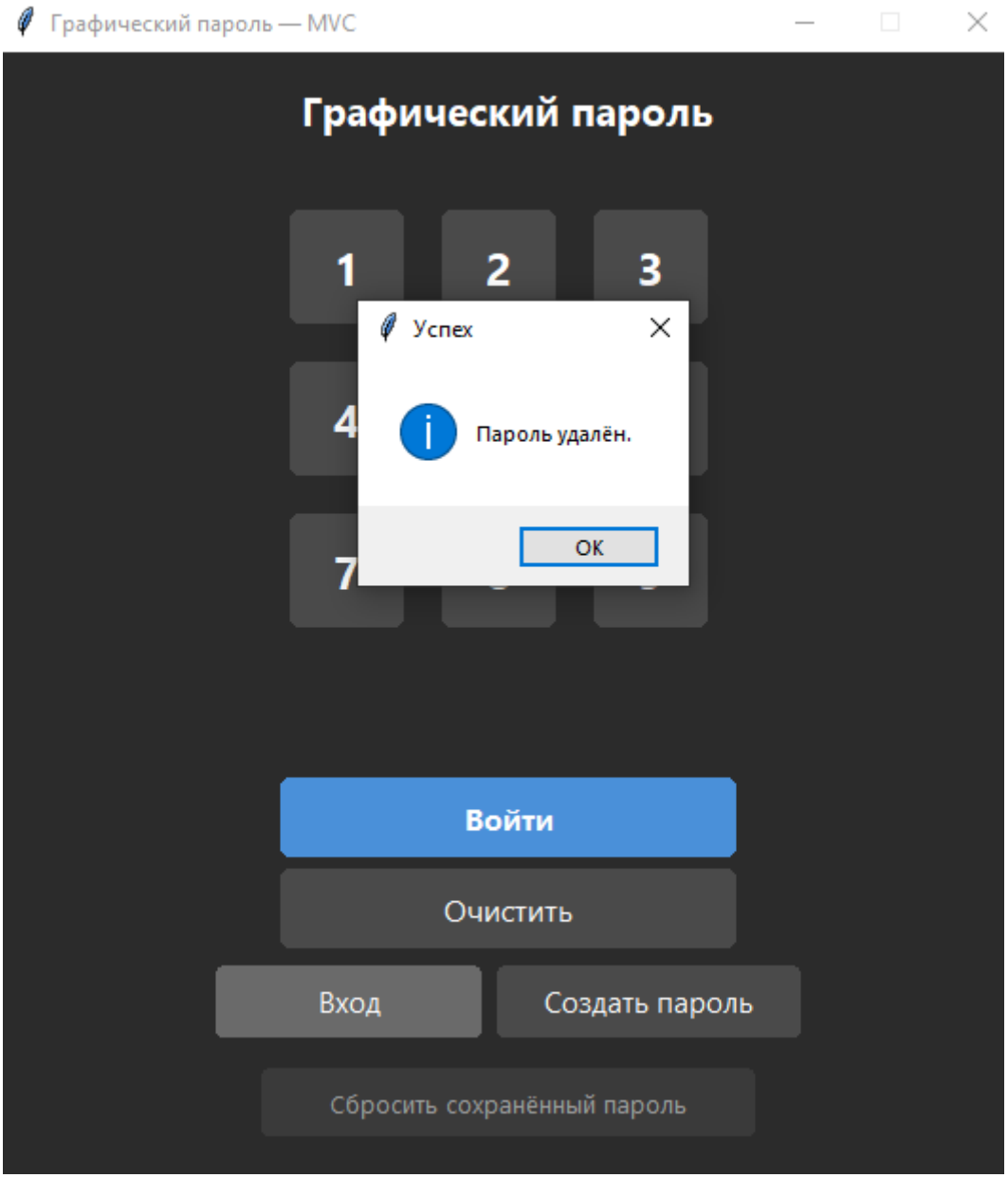


Рисунок 7 - Сброс сохранённого пароля

6. Исходный код

Основные элементы программы:

- PasswordModel - Model (данные и логика);
- PasswordView - View (отображение);
- PasswordController - Controller (связующее звено);
- RoundedButton, rounded_rect() - вспомогательные элементы;
- validate_sequence(), hash_sequence() - вспомогательные функции.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Был изучен и реализован архитектурный паттерн MVC (Model-View-Controller). Model (PasswordModel) содержит данные и бизнес-логику и не зависит от UI. View (PasswordView) отвечает только за отображение и передаёт события в Controller. Controller (PasswordController) связывает Model и View, обрабатывает действия пользователя и обновляет интерфейс. В результате получена работоспособная программа с чётким разделением ответственности между компонентами, что упрощает поддержку и расширение.